

01-STL基础

01-STL基础

一、本阶段目标

这一阶段的目标不是“学算法”。

而是：

- 学会写规范的 C++ 代码
- 熟悉常用 STL 容器
- 建立 CSP 写题的基本手感
- 从“纯语法”过渡到“数据结构思维”

这是整个 CSP 学习的地基。

后面的排序、贪心、二分、图论、DP，几乎都会依赖 STL。

二、知识点总览

本专题主要学习：

- vector 动态数组
 - map 映射
 - set 集合
 - pair 二元组
 - struct 结构体
 - 基础遍历
 - count/find 的使用
 - STL 与算法题结合
-

三、vector 动态数组

1. vector 是什么？

`vector` 可以理解为：

相比普通数组：

- 不需要提前确定大小
- 可以动态添加元素
- 更适合 CSP 写题

2. vector 的定义

```
vector<int> a;
```

定义一个 int 类型动态数组。

3. 常用操作

push_back

向尾部添加元素。

```
a.push_back(5);
```

size

获取数组长度。

```
a.size()
```

下标访问

```
a[i]
```

与普通数组相同。

4. 常见遍历

```
for(int i=0;i<a.size();i++)
{
    cout<<a[i]<<" ";
}
```

这是 CSP 最常用写法。

5. vector 输入模板

```
#include<iostream>
#include<vector>
using namespace std;

int main()
{
    int n;
    cin>>n;

    vector<int> a(n);

    for(int i=0;i<n;i++)
    {
        cin>>a[i];
    }

    for(int i=0;i<n;i++)
    {
        cout<<a[i]<<" ";
    }

    return 0;
}
```

6. 二维 vector 的定义与 push_back

二维 `vector` 可以理解成：

一个 `vector` 里面，每个元素又是一个 `vector`。

也就是：

```
vector<vector<int>>> a;
```

情况 1：一开始就知道行数和列数

如果知道有 `n` 行 `m` 列：

```
vector<vector<int>>> a(n, vector<int>(m));
```

访问方式：

```
a[i][j]
```

输入模板：

```
#include<iostream>
#include<vector>
using namespace std;

int main()
{
    int n,m;
    cin>>n>>m;

    vector<vector<int>>> a(n, vector<int>(m));

    for(int i=0; i<n; i++)
    {
        for(int j=0; j<m; j++)
        {
            cin>>a[i][j];
        }
    }

    return 0;
}
```

情况 2：已经有二维 vector，给某一行 push_back

```
vector<vector<int>>> a(n);
```

这表示：

```
有 n 行，但每一行一开始都是空的
```

可以这样给第 `i` 行加元素：

```
a[i].push_back(x);
```

完整例子：

```
#include<iostream>
#include<vector>
using namespace std;

int main()
{
    int n,m,x;
    cin>>n>>m;

    vector<vector<int>> a(n);

    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            cin>>x;
            a[i].push_back(x);
        }
    }

    return 0;
}
```

情况 3：一整行 push_back 进去

如果想先定义一行，再把这一整行放进二维 vector：

```
vector<vector<int>> a;
vector<int> row;

row.push_back(1);
row.push_back(2);
row.push_back(3);

a.push_back(row);
```

此时：

```
a 里面有 1 行
这一行是：1 2 3
```

情况 4：边读边加入每一行

```
#include<iostream>
#include<vector>
using namespace std;

int main()
{
    int n,m,x;
    cin>>n>>m;

    vector<vector<int>>> a;

    for(int i=0;i<n;i++)
    {
        vector<int> row;

        for(int j=0;j<m;j++)
        {
            cin>>x;
            row.push_back(x);
        }

        a.push_back(row);
    }

    return 0;
}
```

这个写法的逻辑是：

```
先建一行 row
把这一行的 m 个数放进 row
再把 row 放进 a
```

情况 5：二维 vector 加一行空数组

```
vector<vector<int>>> a;

a.push_back(vector<int>());
```

然后可以继续往第 0 行加元素：

```
a[0].push_back(10);  
a[0].push_back(20);
```

情况 6：二维 vector 加一行固定内容

```
vector<vector<int>>> a;  
  
a.push_back(vector<int>{1,2,3});
```

此时：

```
a[0][0] = 1  
a[0][1] = 2  
a[0][2] = 3
```

7. 二维 vector 容易犯的错误

错误 1：没有行，直接访问

错误写法：

```
vector<vector<int>>> a;  
  
a[0].push_back(1);
```

原因：

a 里面还没有第 0 行

正确写法 1：

```
vector<vector<int>>> a(1);  
  
a[0].push_back(1);
```

正确写法 2：

```
vector<vector<int>>> a;
```

```
a.push_back(vector<int>());  
a[0].push_back(1);
```

错误 2：有行但没有列，直接访问列

错误写法：

```
vector<vector<int>> a(3);  
  
a[0][0]=1;
```

原因：

a 有 3 行，但每一行都是空的

正确写法 1：

```
vector<vector<int>> a(3, vector<int>(4));  
  
a[0][0]=1;
```

正确写法 2：

```
vector<vector<int>> a(3);  
  
a[0].push_back(1);
```

错误 3：push_back 和下标赋值混用时没想清楚

如果这样定义：

```
vector<int> a;
```

不能直接：

```
a[0]=5;
```

必须：


```
a.push_back(5);
```

但如果这样定义：

```
vector<int> a(10);
```

就可以：

```
a[0]=5;
```

核心区别是：

push_back: 增加新元素
a[i]=x: 修改已经存在的元素

8. vector 常用初始化方式

一维 vector

```
vector<int> a;
```

空数组。

```
vector<int> a(n);
```

长度为 `n`，初始值默认是 0。

```
vector<int> a(n,5);
```

长度为 `n`，每个元素都是 5。

二维 vector

```
vector<vector<int>>> a;
```

空二维数组。

```
vector<vector<int>> a(n);
```

有 `n` 行，每一行是空的。

```
vector<vector<int>> a(n, vector<int>(m));
```

有 `n` 行 `m` 列，每个元素默认是 0。

```
vector<vector<int>> a(n, vector<int>(m, 5));
```

有 `n` 行 `m` 列，每个元素都是 5。

四、map 映射

1. map 是什么？

`map` 本质是：

| 键值对

例如：

```
名字 -> 分数
字符串 -> 次数
数字 -> 是否出现
```

2. map 的定义

```
map<string, int> mp;
```

表示：

```
字符串 -> int
```

3. map 的核心用途

`map` 最常用于：

- 统计次数
- 判重
- 建立对应关系

4. 插入元素

```
mp["apple"]=3;
```

表示：

```
apple -> 3
```

5. 次数统计

```
mp[s]++;
```

含义：

如果不存在：
自动创建并初始化为 0

然后 +1

这是 map 最重要的功能之一。

6. 判重

count

```
mp.count(x)
```

返回：

- 1: 存在
 - 0: 不存在
-

示例

```
if(mp.count(name))
{
    cout<<"存在";
}
```

7.mp.count() 和 mp[] 的区别

mp.count(name)

作用：

判断是否存在

不会修改 map。

mp[name]

作用：

访问对应值

如果不存在：

会自动创建

这个特别重要。

8. 次数统计模板

```
#include<iostream>
#include<map>
using namespace std;

int main()
{
    int n;
    cin>>n;
```

```
map<string,int> mp;

string s;

for(int i=0;i<n;i++)
{
    cin>>s;
    mp[s]++;
}

for(auto x:mp)
{
    cout<<x.first<<" "<<x.second<<"\n";
}

return 0;
}
```

五、set 集合

1.set 是什么？

set :

| 自动去重 + 自动排序

2.set 的定义

```
set<int> s;
```

3. 插入元素

```
s.insert(5);
```

4. 特点

如果重复插入：

```
5
5
5
```

set 中仍然只有一个 5。

5. 判重

```
if(s.count(x))
{
    cout<<"存在";
}
```

6. 去重模板

```
#include<iostream>
#include<set>
using namespace std;

int main()
{
    int n,x;
    cin>>n;

    set<int> s;

    for(int i=0;i<n;i++)
    {
        cin>>x;
        s.insert(x);
    }

    for(auto x:s)
    {
        cout<<x<<" ";
    }

    return 0;
}
```

六、pair 二元组

1. pair 是什么？

pair 用来：

| 同时存两个值

例如：

姓名 + 分数
坐标 x + y

2. 定义

```
pair<int,int> p;
```

3. 使用

```
p.first=1;  
p.second=2;
```

4. 常见用途

坐标

```
pair<int,int> node;
```

后面 BFS 会大量使用。

七、struct 结构体

1. struct 是什么？

结构体：

2. 定义

```
struct student
{
    string name;
    int score;
};
```

3. 使用

```
student s;

s.name="Tom";
s.score=95;
```

4. 后续用途

后面会大量用于：

- 排序
- 贪心
- 图论
- 模拟

八、STL 的核心思想

很多初学者会觉得：

```
STL = 语法
```

其实不是。

真正重要的是：

“我需要什么功能？”

再去选择：

- vector
- map
- set

常见对应关系

需要动态数组

用：

vector

需要统计次数

用：

map

需要自动去重

用：

set

九、容易犯的错误

1. vector 越界

错误：

```
vector<int> a;
```

```
a[0]=5;
```

因为没有空间。

正确：

```
a.push_back(5);
```

或者：

```
vector<int> a(10);
```

2. map 自动创建

很多人不知道：

```
mp[x]
```

不存在时会自动创建。

容易导致：

- 数据变多
 - 判重错误
-

3. set 不会保留重复元素

很多人会误以为：

```
set 可以统计次数
```

其实不行。

统计次数应该用：

```
map
```

十、STL 边角知识补充

1. vector 的 resize

`resize` 可以改变 vector 的长度。

```
vector<int> a;  
  
a.resize(5);
```

这时 `a` 里面有 5 个元素，默认都是 0。

所以可以这样写：

```
a[0]=10;
```

2. push_back 和 resize 的区别

```
vector<int> a;  
  
a.push_back(1);  
a.push_back(2);
```

表示不断往后加元素。

```
vector<int> a;  
  
a.resize(2);  
a[0]=1;  
a[1]=2;
```

表示先把空间开好，再用下标赋值。

核心区别：

```
push_back: 增加新元素  
resize: 改变已有长度  
a[i]=x: 修改已经存在的位置
```

3. vector 的 clear

```
a.clear();
```

作用：

清空所有元素

清空后：

```
a.size()
```

变成 0。

4. vector 的 empty

```
if(a.empty())  
{  
    cout<<"空的";  
}
```

等价于：

```
if(a.size()==0)
```

5. vector 的 back

```
a.back()
```

表示最后一个元素。

例如：

```
vector<int> a;  
  
a.push_back(3);  
a.push_back(5);  
  
cout<<a.back();
```

输出：

5

注意：

使用 `back` 前，要保证 `vector` 不是空的

6. `vector` 删除最后一个元素

```
a.pop_back();
```

作用：

删除最后一个元素

注意：

使用 `pop_back` 前，也要保证 `vector` 不是空的

7. `map` 会按 `key` 自动排序

```
map<int, int> mp;  
  
mp[3]=1;  
mp[1]=1;  
mp[2]=1;
```

遍历时顺序是：

```
1  
2  
3
```

不是插入顺序。

8. `map` 的遍历

```
for(auto x:mp)
{
    cout<<x.first<<" "<<x.second<<"\n";
}
```

其中：

x.first 表示 key
x.second 表示 value

9. map 删除元素

```
mp.erase(key);
```

例如：

```
mp.erase("Tom");
```

10. map 判断是否为空

```
if(mp.empty())
{
    cout<<"map 是空的";
}
```

11. set 会自动排序

```
set<int> s;

s.insert(3);
s.insert(1);
s.insert(2);
```

遍历时顺序是：

```
1 2 3
```

12. set 的遍历

```
for(auto x:s)
{
    cout<<x<<" ";
}
```

13. set 删除元素

```
s.erase(x);
```

14. set 不能统计次数

如果需要统计出现次数，不能用：

```
set<int> s;
```

应该用：

```
map<int,int> mp;
```

例如：

```
mp[x]++;
```

15. pair 的 first 和 second

```
pair<int,int> p;

p.first=1;
p.second=2;
```

可以理解为：

```
p = (1,2)
```

16. pair 的初始化

```
pair<int,int> p={1,2};
```

或者：

```
pair<int,int> p=make_pair(1,2);
```

17. pair 的排序规则

如果有：

```
vector<pair<int,int>> a;
```

直接排序：

```
sort(a.begin(),a.end());
```

排序规则是：

```
先按 first 从小到大排
如果 first 相同，再按 second 从小到大排
```

18. struct 配合 sort

```
#include<iostream>
#include<algorithm>
using namespace std;

struct student
{
    string name;
    int score;
};
```



```

bool cmp(student a, student b)
{
    return a.score > b.score;
}

int main()
{
    int n;
    cin >> n;

    student a[100];

    for(int i=0; i<n; i++)
    {
        cin >> a[i].name >> a[i].score;
    }

    sort(a, a+n, cmp);

    for(int i=0; i<n; i++)
    {
        cout << a[i].name << " " << a[i].score << "\n";
    }

    return 0;
}

```

19. auto 的作用

`auto` 可以让编译器自动判断类型。

例如：

```

for(auto x:s)
{
    cout << x << " ";
}

```

不用手动写：

```

set<int>::iterator

```

现阶段记住：

遍历 STL 时, auto 很方便

20. 使用 STL 要加对应头文件

```
#include<vector>
#include<map>
#include<set>
#include<algorithm>
```

分别对应：

```
vector
map
set
sort
```

如果使用万能头：

```
#include<bits/stdc++.h>
```

就基本都包含了。

但你平时也可以继续按专题写具体头文件。

21. STL 常见选择表

需求	推荐工具
存一串数	vector
存二维矩阵	vector<vector>
统计次数	map
判断是否出现过	set / map
自动去重	set
存两个相关值	pair
存多个相关值	struct
排序	sort